

Python Dictionaries ¶

Dictionary

Dictionaries are used to store data values in key:value pairs.

A dictionary is a collection which is ordered, changeable and do not allow duplicates.

Dictionaries are written with curly brackets, and have keys and values:

In []: *# Create and print a dictionary:*

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
print(thisdict)
```

Dictionary Items

Dictionary items are ordered, changeable, and do not allow duplicates.

Dictionary items are presented in key:value pairs, and can be referred to by using the key name.

In []: *# Print the "brand" value of the dictionary:*

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
print(thisdict["brand"])
```

Ordered or Unordered?

When we say that dictionaries are ordered, it means that the items have a defined order, and that order will not change.

Unordered means that the items do not have a defined order, you cannot refer to an item by using an index.

Changeable

Dictionaries are changeable, meaning that we can change, add or remove items after the dictionary has been created.

Duplicates Not Allowed

Dictionaries cannot have two items with the same key:

```
In [ ]: # Duplicate values will overwrite existing values:

thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964,
    "year": 2020
}
print(thisdict)
```

Dictionary Length

To determine how many items a dictionary has, use the len() function:

```
In [ ]: # Print the number of items in the dictionary:

print(len(thisdict))
```

Dictionary Items - Data Types

The values in dictionary items can be of any data type:

```
In [ ]: ### String, int, boolean, and list data types:

thisdict = {
    "brand": "Ford",
    "electric": False,
    "year": 1964,
    "colors": ["red", "white", "blue"]
}
print(thisdict)
```

type()

From Python's perspective, dictionaries are defined as objects with the data type 'dict':

<class 'dict'>

```
In [ ]: # Print the data type of a dictionary:

thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
print(type(thisdict))
```

The dict() Constructor

It is also possible to use the dict() constructor to make a dictionary.

```
In [ ]: #Using the dict() method to make a dictionary:

thisdict = dict(name = "John", age = 36, country = "Norway")
print(thisdict)
```

Python Collections (Arrays)

There are four collection data types in the Python programming language:

List is a collection which is ordered and changeable. Allows duplicate members.

Tuple is a collection which is ordered and unchangeable. Allows duplicate members.

Set is a collection which is unordered, unchangeable*, and unindexed. No duplicates

Accessing Items

You can access the items of a dictionary by referring to its key name, inside square brackets:

```
In [ ]: # Get the value of the "model" key:

thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
x = thisdict["model"]
print(x)
```

There is also a method called `get()` that will give you the same result:

```
In [ ]: # Get the value of the "model" key:

x = thisdict.get("model")
print(x)
```

Get Keys

The `keys()` method will return a list of all the keys in the dictionary.

```
In [ ]: # Get a list of the keys:

x = thisdict.keys()
print(x)
```

The list of the keys is a view of the dictionary, meaning that any changes done to the dictionary will be reflected in the keys list.

```
In [ ]: # Add a new item to the original dictionary, and see that the keys list gets u

car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

x = car.keys()

print(x) #before the change

car["color"] = "white"

print(x) #after the change
```

Get Values

The `values()` method will return a list of all the values in the dictionary.

```
In [ ]: # Get a list of the values:

x = thisdict.values()
print(x)
```

The list of the values is a view of the dictionary, meaning that any changes done to the dictionary will be reflected in the values list.

```
In [ ]: # Make a change in the original dictionary, and see that the values list gets

car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

x = car.values()

print(x) #before the change

car["year"] = 2020

print(x) #after the change
```

```
In [ ]: # Add a new item to the original dictionary, and see that the values list gets

car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

x = car.values()

print(x) #before the change

car["color"] = "red"

print(x) #after the change
```

Get Items

The `items()` method will return each item in a dictionary, as tuples in a list.

```
In [ ]: # Get a list of the key:value pairs

x = thisdict.items()
print(x)
```

The returned list is a view of the items of the dictionary, meaning that any changes done to the dictionary will be reflected in the items list.

```
In [ ]: # Make a change in the original dictionary, and see that the items list gets u

car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

x = car.items()

print(x) #before the change

car["year"] = 2020

print(x) #after the change
```

```
In [ ]: # Add a new item to the original dictionary, and see that the items list gets

car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

x = car.items()

print(x) #before the change

car["color"] = "red"

print(x) #after the change
```

Check if Key Exists

To determine if a specified key is present in a dictionary use the **in** keyword:

```
In [ ]: # Check if "model" is present in the dictionary:

thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
if "model" in thisdict:
    print("Yes, 'model' is one of the keys in the thisdict dictionary")
```

Python - Change Dictionary Items

Change Values

You can change the value of a specific item by referring to its key name:

```
In [ ]: # Change the "year" to 2018:

thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
thisdict["year"] = 2018
print(thisdict)
```

Update Dictionary

The `update()` method will update the dictionary with the items from the given argument.

The argument must be a dictionary, or an iterable object with **key:value** pairs.

```
In [ ]: # Update the "year" of the car by using the update() method:
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.update({"year": 2020})  
print(thisdict)
```

Python - Add Dictionary Items

Adding Items

Adding an item to the dictionary is done by using a new index key and assigning a value to it:

```
In [ ]: thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict["color"] = "red"  
print(thisdict)
```

Update Dictionary

The `update()` method will update the dictionary with the items from a given argument. If the item does not exist, the item will be added.

The argument must be a dictionary, or an iterable object with **key:value** pairs.

```
In [ ]: # Add a color item to the dictionary by using the update() method:
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.update({"color": "red"})  
print(thisdict)
```


Python - Remove Dictionary Items

Removing Items

There are several methods to remove items from a dictionary:

The pop() method removes the item with the specified key name:



```
In [ ]: thisdict = {  
        "brand": "Ford",  
        "model": "Mustang",  
        "year": 1964  
    }  
thisdict.pop("model")  
print(thisdict)
```

The popitem() method removes the last inserted item

```
In [ ]: thisdict = {  
        "brand": "Ford",  
        "model": "Mustang",  
        "year": 1964  
    }  
thisdict.popitem()  
print(thisdict)
```

The del keyword removes the item with the specified key name

```
In [ ]: thisdict = {  
        "brand": "Ford",  
        "model": "Mustang",  
        "year": 1964  
    }  
del thisdict["model"]  
print(thisdict)
```

The del keyword can also delete the dictionary completely:

```
In [ ]: thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
del thisdict
print(thisdict) #this will cause an error because "thisdict" no longer exists.
```

The clear() method empties the dictionary:

```
In [ ]: thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
thisdict.clear()
print(thisdict)
```

Python - Loop Dictionaries

Loop Through a Dictionary

You can loop through a dictionary by using a for loop.

When looping through a dictionary, the return value are the keys of the dictionary, but there are methods to return the values as well.



```
In [ ]: # Print all key names in the dictionary, one by one:
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
for x in thisdict:
    print(x)
```

```
In [ ]: # Print all values in the dictionary, one by one:
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
for x in thisdict:
    print(thisdict[x])
```

```
In [ ]: # You can also use the values() method to return values of a dictionary:
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
for x in thisdict.values():
    print(x)
```

```
In [ ]: # You can use the keys() method to return the keys of a dictionary:

thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
for x in thisdict.keys():
    print(x)
```

```
In [ ]: # Loop through both keys and values, by using the items() method:
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
for x, y in thisdict.items():
    print(x, y)
```

Python - Copy Dictionaries

Copy a Dictionary

You cannot copy a dictionary simply by typing `dict2 = dict1`, because: `dict2` will only be a reference to `dict1`, and changes made in `dict1` will automatically also be made in `dict2`.

There are ways to make a copy, one way is to use the built-in Dictionary method `copy()`.

```
In [ ]: # Make a copy of a dictionary with the copy() method:
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
mydict = thisdict.copy()  
print(mydict)
```

Another way to make a copy is to use the built-in function dict().

```
In [ ]:
```

```
# Make a copy of a dictionary with the dict() function:
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
mydict = dict(thisdict)  
print(mydict)
```

Python - Nested Dictionaries

Nested Dictionaries

A dictionary can contain dictionaries, this is called nested dictionaries.

```
In [ ]:
```

```
# Create a dictionary that contain three dictionaries:
```

```
myfamily = {  
    "child1" : {  
        "name" : "Emil",  
        "year" : 2004  
    },  
    "child2" : {  
        "name" : "Tobias",  
        "year" : 2007  
    },  
    "child3" : {  
        "name" : "Linus",  
        "year" : 2011  
    }  
}  
print(myfamily)
```

Or, if you want to add three dictionaries into a new dictionary:

In []:

```
# Create three dictionaries, then create one dictionary that will contain the

child1 = {
    "name" : "Emil",
    "year" : 2004
}
child2 = {
    "name" : "Tobias",
    "year" : 2007
}
child3 = {
    "name" : "Linus",
    "year" : 2011
}

myfamily = {
    "child1" : child1,
    "child2" : child2,
    "child3" : child3
}
print(myfamily)
```

Access Items in Nested Dictionaries

To access items from a nested dictionary, you use the name of the dictionaries, starting with the outer dictionary:

In []:

```
# Print the name of child 2:

print(myfamily["child2"]["name"])
```

Loop Through Nested Dictionaries

You can loop through a dictionary by using the `items()` method like this:

In []:

```
# Loop through the keys and values of all nested dictionaries:

for x, obj in myfamily.items():
    print(x)

    for y in obj:
        print(y + ': ', obj[y])
```

Python Dictionary Methods

Python has a set of built-in methods that you can use on dictionaries.

Python Dictionary Methods

Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key-value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair (pre-Python 3.7 behavior varies)
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value.
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary